AI Forward Deployed Engineer

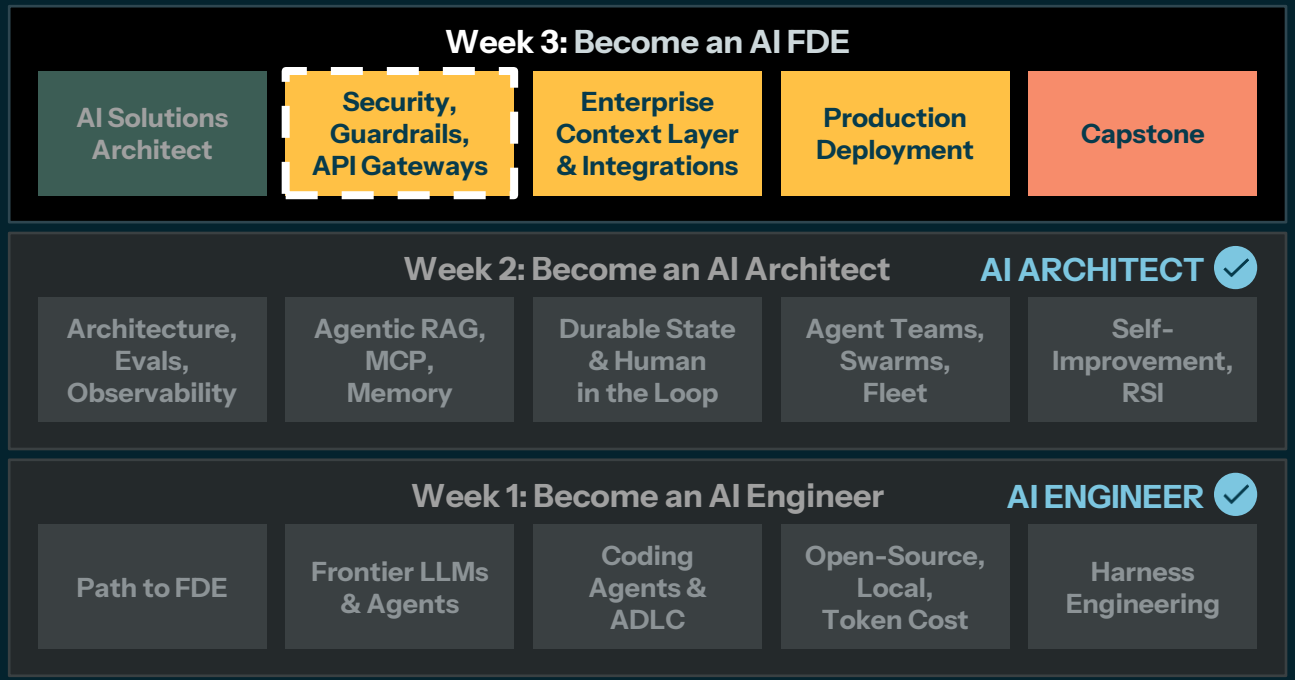
The AI Architect & FDE Course



Week 3 Day 2

3 weeks to Forward Deployed Engineer

Theory Practical Projects



Taking stock

You can already:

- ✓ Deliver solutions that measurably solve business problems with evals and observability
- ✓ Create durable and scalable agents in LangGraph with a human in the loop
- ✓ Engineer an Agent in a Harness and use MCP and Agentic RAG
- ✓ Drive an AI implementation from discovery to delivery and maintenance

By the end of today, you'll be able to:

- ○ Identify and mitigate security risks in agents including the lethal trifecta
- ○ Put guardrails on an agent's tools and on the model's input and output
- ○ Explain the role of API, MCP and model gateways in the enterprise
- ○ Deploy an app on AWS Lambda behind an AgentCore Gateway and Policy

Recap

We planned an FDE engagement from discovery to handover



Lifecycle and tollgates

A sponsor signs off each phase



Systems architecture

Gateways and the context layer



How much autonomy

Based on the risk level & risk tolerance



Handover and adoption

The client runs it without the FDE

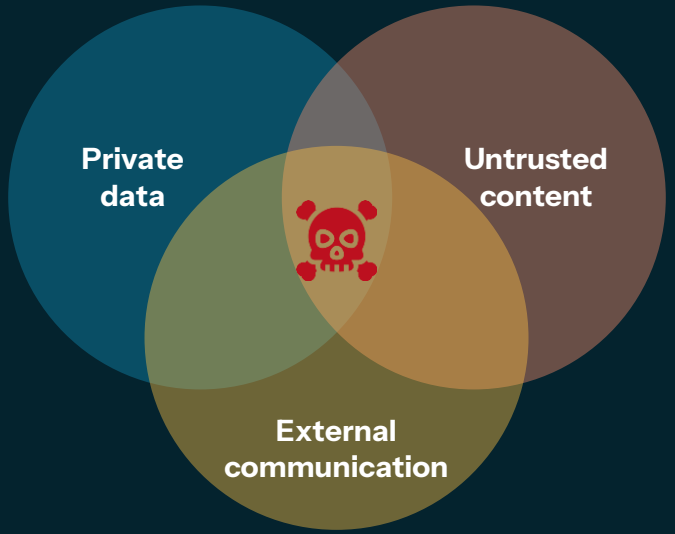
Security – a course of its own

1. **Constrain the agent runtime**
   Permissions and sandboxes
2. **Limit the scope of its APIs**
   Put boundaries on what it can do
3. **Prompt injection attacks**
   Recognize the ‘lethal trifecta’ scenario

Identify risks, manage and mitigate them with monitoring and control

The Lethal Trifecta

Coined by AI luminary Simon Willison



A classic example was the vulnerability identified in Github MCP

Safety

Implement guardrails to ensure LLM responses are as expected



An LLM validator

Use another LLM to check each response before it goes out



Coded checks

Actually check, in code, for problematic words



Flagging in the platform

Give users a button for bad responses, and review them



Comprehensive testing, audits

Before go-live and continuous via observability

API, MCP and model gateways

An architecture investment that makes a lot of sense



API gateway

The front door to a company's APIs

Keys, rate limits and logs

Azure API Management, AWS API Gateway, Kong



MCP gateway

Turns APIs into tools for agents

Who can call which tool

AWS AgentCore Gateway, MuleSoft Agent Fabric



Model gateway

One API in front of many models

Budgets and fallbacks

AWS AgentCore Gateway, OpenRouter, LiteLLM

AI gateways in industry

Ranked by use in AI projects

1. **AWS Bedrock AgentCore Gateway**
   More US job adverts mention AgentCore than Azure API Management or Kong
2. **Azure API Management** Established API platform
   AI gateway in 1,200+ enterprises by Microsoft's count (Nov 2025)
3. **Databricks Unity AI Gateway**
   Thousands of customers by Databricks' count and often on by default
4. **Kong AI Gateway** Established API platform
   Hundreds of AI Gateway customers by Kong's count (Aug 2026)
5. **Prisma AIRS AI Gateway (Portkey)**
   Bought by Palo Alto Networks to secure AI traffic (May 2026)

Early	Apigee, MuleSoft Agent Fabric, Cloudflare AI Gateway, Bifrost
Dev favorites	LiteLLM, OpenRouter, Composio

Gateways side by side

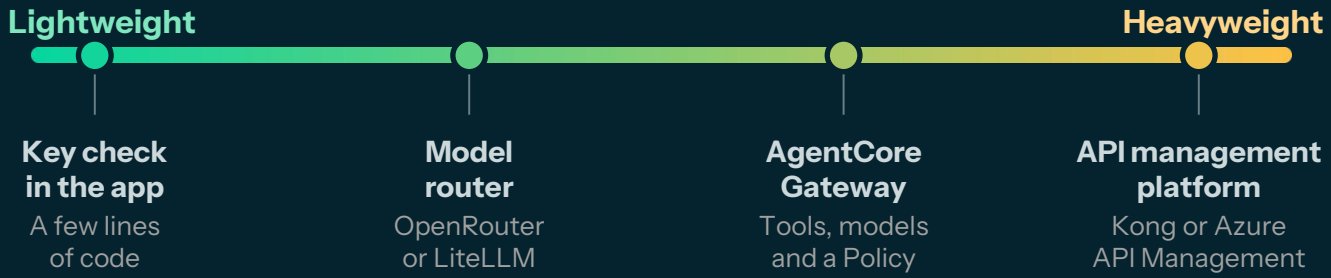
What each one covers

	LLM router	API gateway	MCP gateway External	MCP gateway Internal
AgentCore Gateway	●	○	○	●
Azure API Management	●	●		●
Databricks AI Gateway	●		○	●
Kong	●	●		●
Prisma AIRS (Portkey)	●		○	●
Apigee	●	●		●
MuleSoft	●	●	●	●
Cloudflare	●	○		●
Bifrost	●			●
LiteLLM	●	○		●
OpenRouter	●			
Composio			●	○

● Yes ○ Partial: preview, experimental or limited

Lightweight or heavyweight

My advice is to keep it simple until gateways are more stable



Lightweight

Pros
Quick, cheap and easy to replace

Cons
Controls repeated in every app

Heavyweight

Pros
Policy, catalog and audit in one place

Cons
Cost, lock-in and a layer still changing

AWS Bedrock AgentCore

A suite of services for agents

Gateway Turns APIs into MCP tools	**Policy** Checks rules on every tool call	**Identity** Manages agents and API keys	**Observability** Policy decisions in CloudWatch
Memory Long-term memory for agents	**Runtime** Serverless hosting for the agent	**Harness** A managed agent loop from config	**Evaluations** Scores agents from their traces
Optimization Tests suggested prompt changes	**Registry** A catalog of agents and MCP servers	**Code Interpreter** A sandbox where agents run code	**Browser** A cloud browser for agents
Web Search Web search as an MCP tool	**Payments** Lets agents pay for APIs and content		

Week 3 Day 2
Week 3 Day 3
Week 3 Day 4
Others

UnderCover by the end of today

What runs on your laptop and what runs on AWS



Lab 2 notebook
Calls the agent's tools through the Gateway

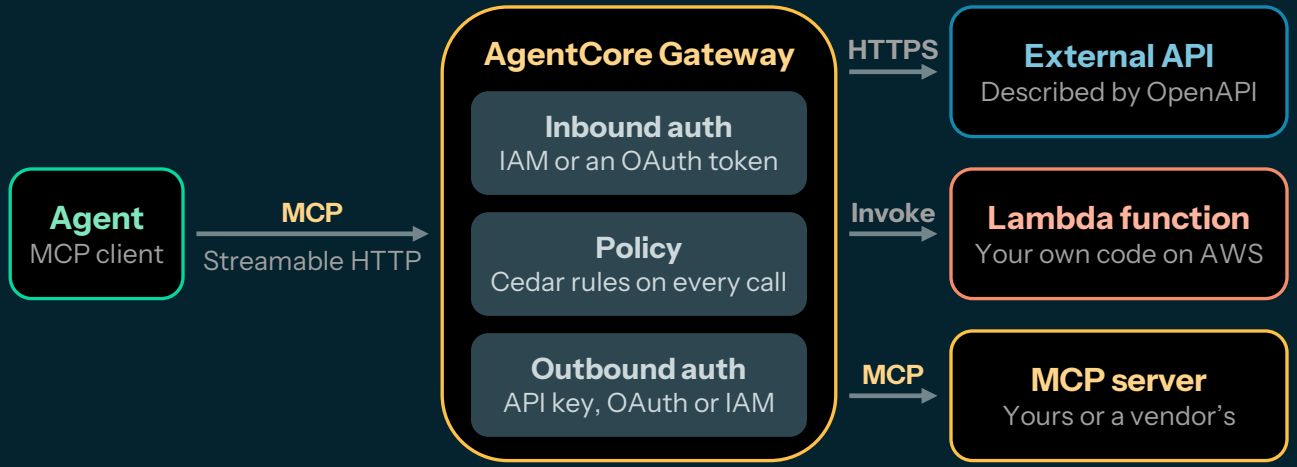
UnderCover app
The API and the dashboard for handlers

Agent
Calls the backend on your laptop with the API key

Supabase
One database for your laptop and Lambda

Calling a tool through the Gateway

The agent talks to the Gateway over MCP



The agent sees one list of tools

Converting an API endpoint into a tool

In our project today, we will make endpoints and provide them as tools

The endpoint in our app

```python
# backend/main.py
@app.post(
    "/api/claims/{claim_id}/payment",
    operation_id="pay_claim",
)
def pay_claim(
    claim_id: str,
    request: PaymentRequest,
):
    """Pay an approved claim its
    payout once, unless it needs a
    handler's sign-off."""
```

The tools for the agent

```
undercover-app___get_claim
undercover-app___get_policy
undercover-app___get_claim_history
undercover-app___search_documents
undercover-app___read_section
undercover-app___calculate_payout
undercover-app___record_decision
undercover-app___pay_claim
  Pay an approved claim its
  payout once, unless it needs
  a handler's sign-off.
  input: claim_id, amount
```

On each call → SigV4 checked → Policy decides → API key added → Runs on Lambda

UnderCover's guardrails on AWS

Three layers keep a payout within the rules



The agent
A notebook today

AgentCore Gateway
Our API as MCP tools

UnderCover app
On Lambda with Supabase

Model check
What goes into the model or what comes out of it

Cedar Policy
No payout over $1,000
Only the tools it needs

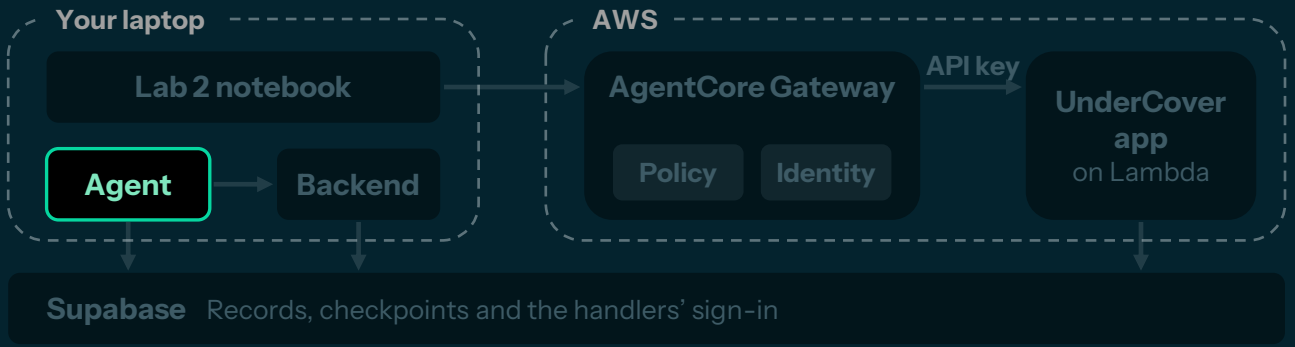
API key and limit
Calls need the API key
A second $1,000 limit

The Policy in Cedar
Log only first then enforced

```
forbid(
  principal is AgentCore::IamEntity,
  action == AgentCore::Action::"undercover-app___pay_claim",
  resource == AgentCore::Gateway::"<gateway arn>"
) when {
  context.input.amount.greaterThan(decimal("1000.00"))
};
```

Lab 1: a guardrail on the model

Catching an injected instruction before the agent acts on it



1. **The injected claim**
   CLM-1901 says to approve it in full
2. **A model that falls for it**
   We try models until one approves it
3. **A keyword check**
   A reworded instruction gets past it
4. **A model check as middleware**
   A cheap model reads the documents
5. **Refer it to a handler**
   Sent to a person with the reason

Upgrading the product

Four changes before the app goes on the internet



1. **An API key for the agent**
   Every route it calls checks X-API-Key
2. **The model check**
   Lab 1's middleware in the claim agent
3. **The agent's own API document**
   The gateway turns an API + docs into MCP
4. **Ready for Lambda**
   The app and dashboard in one package

Building it with Claude Code

The change is written up in labs/week3day2_product_SECURE.md

1. **Copy the brief into docs**
   `cp labs/week3day2_product_SECURE.md docs/SECURE.md`
2. **Ask Claude Code for a plan**
   Please plan the security changes in docs/SECURE.md
3. **Let it build and commit each phase**
   Please implement the plan for the security changes
4. **Check out my version**
   `git checkout week3day2-start` or `git checkout week3day2-end`

Lab 2: the Gateway and the Policy

Deploying the app behind an AgentCore Gateway

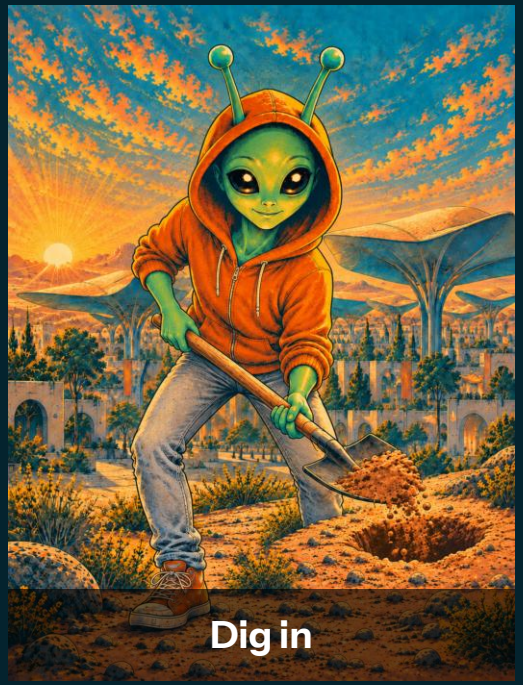


1. **Deploy the app to Lambda**
   Then sign in on the deployed dashboard
2. **Create the Gateway**
   The agent's routes become MCP tools
3. **Call the tools from Python**
   Signed with SigV4 as your IAM user
4. **Add the Policy in log-only mode**
   The app's limit refuses a $1,150 payout
5. **Switch the Policy to enforce**
   Now the Gateway refuses the payout
6. **Tear down**
   Delete what today's lab created

Two ways to follow Lab 2

Choose how deep to go into the Gateway nuts and bolts like Cedar





UnderCover by the end of Week 3

Where each piece runs once the agent is deployed on Day 4



Models still come from OpenRouter and traces still go to Langfuse

3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

AI ARCHITECT

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer

AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course

